Department of AI and Data Science Engineering

Introduction to Industrial Internet of Things
CIA 2 Report

Intelligent Waste Management And Bin Fill-Level Prediction

Class - 6BTCS AIML C

Apoorv Tiwari- 2362037
Arnav Jain- 2362039
Arun Johny-2362040
Barath NS-2362050

Github Link- apoorv-tiwari0/Waste-Bin-Monitor

School of Engineering and Technology,
CHRIST (Deemed to be University),
Kumbalagodu, Bengaluru-560 074
February 2026

1. Project Overview

The IoT Waste Bin Monitor is an embedded systems project that uses a NodeMCU ESP8266 microcontroller to continuously track the fill level of a waste bin and detect object presence at its mouth. Real-time sensor data is displayed locally on a 0.96" OLED screen and transmitted to the ThingSpeak IoT cloud platform every 15 seconds for remote monitoring and analytics.

The system uses two sensing modalities working together:

- HC-SR04 Ultrasonic Sensor — measures the distance from the sensor (mounted at the top of the bin) to the waste surface to compute the fill percentage.
- IR Proximity Sensor — detects whether an object or hand is present at the bin opening, enabling lid-open or presence detection.
- SSD1306 OLED Display — provides real-time local visual feedback: fill %, visual fill bar, distance reading, and IR status.
- ESP8266 Wi-Fi — transmits all sensor data to ThingSpeak cloud for remote dashboarding.

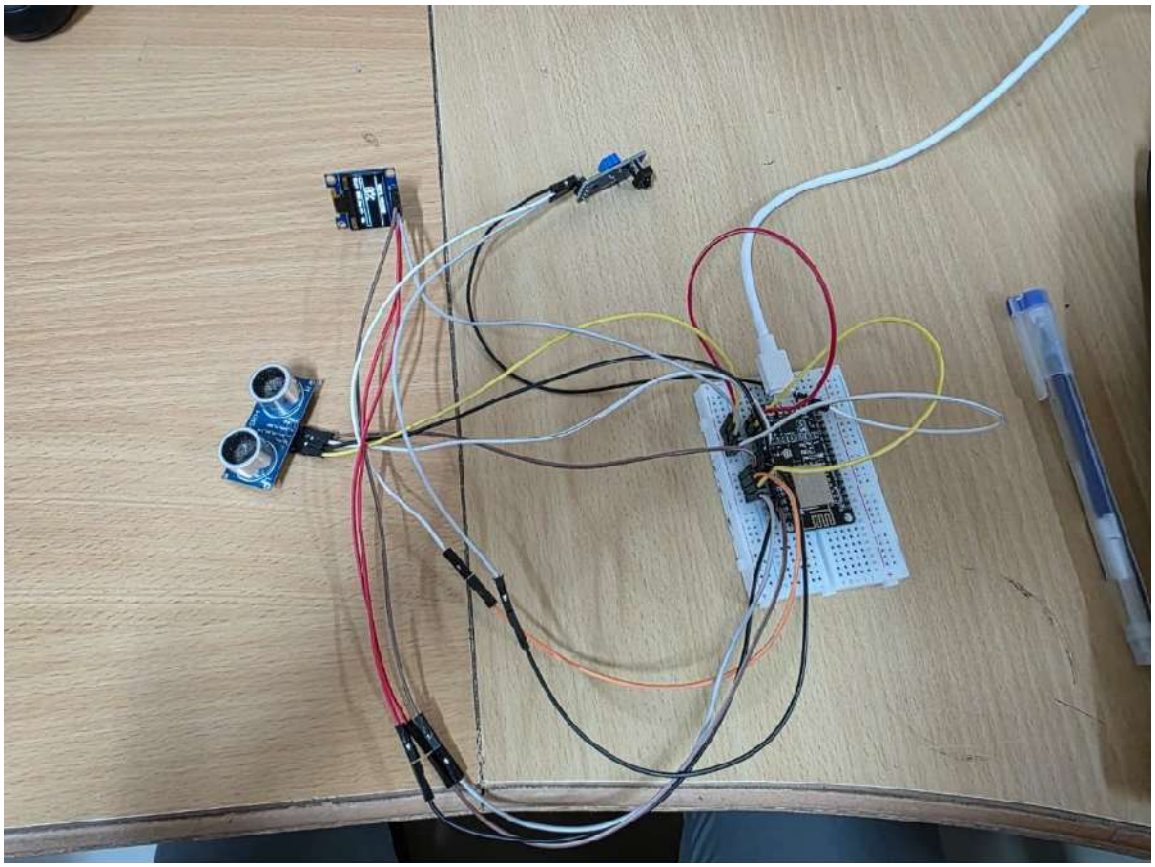


Figure 1: Assembled Hardware — NodeMCU on breadboard, HC-SR04 (top), IR Sensor (right), and OLED Display (top-right)

2. Hardware Components

2.1 NodeMCU v1.0 — ESP8266 ESP-12E (Main Controller)

The NodeMCU v1.0 is a Wi-Fi capable microcontroller development board built around the Espressif ESP8266 ESP-12E module. It serves as the brain of the system — reading sensors, computing fill levels, driving the display, and pushing data to the cloud.

Role & Functionality in This Project

The NodeMCU is the central processing unit that ties every component together. It has five distinct responsibilities in this system:

- Sensor Controller — It generates the 10 µs TRIG pulse for the HC-SR04, measures the returning ECHO pulse width using pulseIn(), and polls the IR sensor digital pin every loop cycle (~1 second).
- Signal Processor — It runs the distance-to-fill-percentage conversion formula in software and applies edge-case constraints (0–100%) so that noisy or out-of-range readings do not corrupt the output.
- Display Driver — Via the I2C bus (SDA/SCL), it sends pixel-level drawing commands to the OLED every second, rendering the fill bar, large percentage text, distance, and IR status.
- Wi-Fi Client & IoT Publisher — The built-in ESP8266 Wi-Fi module connects to the local network and makes HTTP POST requests to ThingSpeak every 15 seconds, sending 3 data fields per upload.
- Serial Debug Interface — It outputs all live readings at 115200 baud over USB-serial so the developer can monitor the system in real time via the Arduino IDE Serial Monitor.

Why NodeMCU was chosen: It is one of the most cost-effective boards that combines a capable 32-bit MCU with onboard Wi-Fi, making it ideal for IoT projects that need both local sensing/processing AND cloud connectivity — without requiring any external Wi-Fi shield or module. The CH340G USB-serial chip also allows direct programming from Arduino IDE without external programmers.

Key Technical Specifications:

- CPU: Tensilica L106 32-bit RISC processor @ 80 MHz (overclockable to 160 MHz)
- Wi-Fi: IEEE 802.11 b/g/n; supports Station (STA), Access Point (AP), and STA+AP modes
- Operating Voltage: 3.3V logic; 5V input via USB or VIN pin
- Flash Memory: 4 MB (SPI Flash)
- RAM: 64 KB instruction RAM, 96 KB data RAM
- GPIO Pins: 11 usable digital I/O pins (D0-D10), 1 analog input (A0, 0-1V range)
- Interfaces: UART (serial), I2C, SPI, PWM
- USB-to-Serial Chip: CH340G (onboard, for programming via Arduino IDE)
- Onboard LED: Active LOW on GPIO2 (D4)

Complete Pin Reference Table

The table below covers every pin on the NodeMCU board, its function, and how it is utilized in this project:

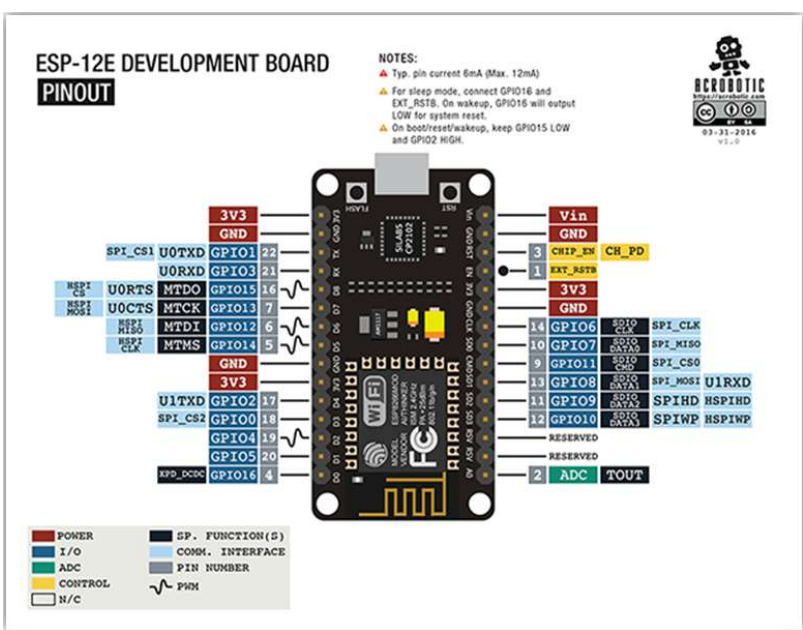
Pin / Label	Function	Description	Used In Project
RST	Reset	Resets the module when pulled LOW	Not connected
ADC0 (A0)	Analog Input	Single analog input (0-1V range)	Not used
EN	Enable	Enables chip when HIGH	Pulled HIGH (internal)
IO16 (D0)	GPIO 16	Can wake from deep sleep	Not used
IO14 (D5)	GPIO 14 / SPI CLK	Digital GPIO — IR Sensor digital output	IR Sensor OUT

IO12 (D6)	GPIO 12 / SPI MISO	Digital GPIO — Ultrasonic TRIG	HC-SR04 TRIG
IO13 (D7)	GPIO 13 / SPI MOSI	Digital GPIO — Ultrasonic ECHO	HC-SR04 ECHO
IO15 (D8)	GPIO 15 / SPI CS	Must be LOW at boot	Not used
RXD (D9)	UART RX	Serial receive	Serial communication
TXD (D10)	UART TX	Serial transmit	Serial communication
GND	Ground	Common ground reference	All GND connections
3.3V (3V3)	3.3V Output	Regulated 3.3V supply output	IR sensor VCC, OLED VCC
VIN / VU	USB/Vin Power	5V from USB; main board power	USB power input
IO4 (D2)	GPIO 4 / I2C SDA	I2C Data line	OLED SDA
IO5 (D1)	GPIO 5 / I2C SCL	I2C Clock line	OLED SCL

Pins actively used in this project (highlighted):

- D5 (GPIO14) — IR Sensor Digital Output
- D6 (GPIO12) — HC-SR04 TRIG (Ultrasonic Trigger)
- D7 (GPIO13) — HC-SR04 ECHO (Ultrasonic Echo)
- D1 (GPIO5) — OLED SCL (I2C Clock)
- D2 (GPIO4) — OLED SDA (I2C Data)
- 3.3V + GND — Power rails for sensors and OLED

Important Note on GPIO Voltage: The NodeMCU operates at 3.3V logic, but the HC-SR04 ECHO pin outputs 5V TTL. The ESP8266 GPIO pins have some tolerance but using a voltage divider (1kΩ + 2kΩ) on the ECHO line is recommended for long-term reliability. The TRIG pin only receives 3.3V output from the MCU, which is sufficient to trigger the HC-SR04.



2.2 HC-SR04 Ultrasonic Sensor

The HC-SR04 is an ultrasonic ranging module that can measure distances from 2 cm to 400 cm with high precision. In this project, it is mounted at the top of the bin lid, facing downward, measuring the distance to the waste surface.

Role & Functionality in This Project

The HC-SR04 is the primary measurement sensor responsible for quantifying how full the waste bin is. Here is exactly what it does in context:

- Fill Level Measurement — Mounted at the top of the bin (facing down), it continuously measures the air gap between the sensor and the top of the waste. A 40 cm bin with 5 cm of air gap means the bin is 87.5% full.
- Non-contact Sensing — Unlike a float or pressure sensor, it does not touch the waste at all. This is critical for hygiene and durability — the sensor stays clean regardless of what is in the bin.
- Distance-to-Percentage Conversion Input — The raw distance value (in cm) is fed into the calculateFillPercent() function, which maps the reading to a 0–100% fill scale based on the defined bin height.
- Cloud Data Source — Every 15 seconds, the measured distance is also sent as Field 2 to ThingSpeak alongside the fill percentage, giving the raw sensor data for trend analysis.

Why HC-SR04 was chosen: It is low-cost, widely supported in Arduino libraries, and provides sufficient accuracy (+/-3 mm) for waste-level monitoring where centimeter precision is more than adequate. Its 2–400 cm range covers virtually any bin size. The simple 2-pin (TRIG/ECHO) digital interface makes it straightforward to integrate with a microcontroller.

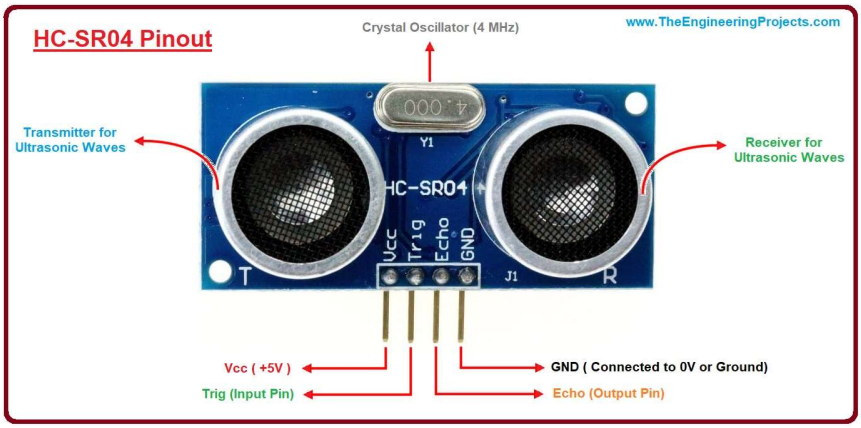
Technical Specifications:

- Operating Voltage: 5V DC
- Operating Current: ~15 mA
- Ultrasonic Frequency: 40 kHz
- Measuring Range: 2 cm to 400 cm
- Accuracy: +/- 3 mm
- Effective Beam Angle: ~15 degrees
- Trigger Input: 10 microsecond TTL HIGH pulse
- Echo Output: TTL HIGH pulse, duration proportional to measured distance
- Minimum Cycle Time: 60 ms (to avoid echo overlap between readings)

Pin Connections:

- VCC — 5V power
- GND — Ground
- TRIG — Trigger input → D6 (GPIO12) on NodeMCU
- ECHO — Echo output → D7 (GPIO13) on NodeMCU (note: 5V output, NodeMCU is 3.3V tolerant via internal protection)

Mounting Consideration: The sensor should be mounted centrally at the top of the bin, perpendicular to the waste surface. Angled mounting causes the echo to reflect away from the sensor, returning an inaccurate reading. Soft materials like cloth may absorb sound and reduce reflectivity, affecting accuracy.



2.3 IR Proximity Sensor (FC-51 Type)

The IR proximity sensor module uses an infrared LED + photodiode pair to detect the presence of objects within a configurable range. In this project it detects whether someone is placing waste into the bin (object at mouth level).

Role & Functionality in This Project

The IR sensor serves a different and complementary purpose to the ultrasonic sensor — it detects events at the bin opening rather than measuring the bin level:

- Bin Usage Detection — Positioned near the mouth of the bin, it detects when a hand or object approaches to deposit waste. This acts like a "bin opened" trigger event.
- Activity Logging — Every IR detection event is logged to ThingSpeak as Field 3 (binary 1 or 0). Over time, this data reveals usage frequency — how often the bin is being used throughout the day.
- Complementary Sensing Layer — While the ultrasonic sensor tells you HOW FULL the bin is, the IR sensor tells you HOW OFTEN the bin is being used. Together they give a complete picture of bin activity.
- Real-time OLED Indicator — The OLED always shows "IR: YES" or "IR: NO" in the bottom-right corner, giving instant visual feedback of current presence detection status.

Why IR was chosen over a second ultrasonic sensor: For simple presence detection at the bin mouth, an IR sensor is cheaper, faster-responding, and sufficient since binary output (detected / not detected) is all that is needed. An ultrasonic sensor at the mouth would be overkill and add unnecessary complexity for a binary use case.

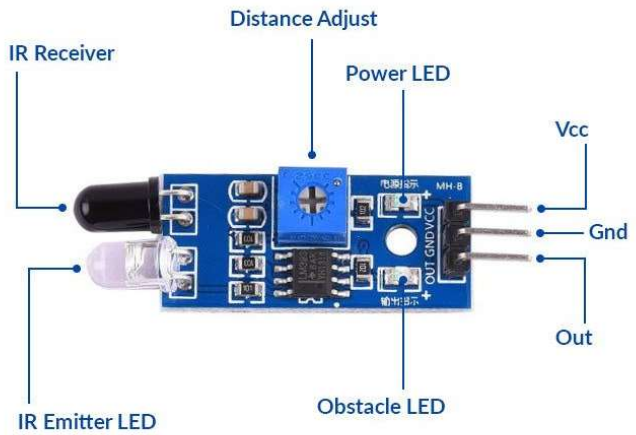
Technical Specifications:

- Operating Voltage: 3.3V to 5V DC
- Detection Range: 2 cm to 30 cm (adjustable via onboard potentiometer)
- Output Type: Digital (Active LOW — goes LOW when object detected)
- IR LED Wavelength: ~940 nm (near infrared, invisible to human eye)
- Comparator IC: LM393 dual comparator (onboard)
- Onboard Indicator: Red power LED + green detection LED
- Response Time: Very fast (microseconds)

Pin Connections:

- VCC — 3.3V from NodeMCU
- GND — Ground
- OUT — Digital output → D5 (GPIO14) on NodeMCU

Calibration Note: The onboard potentiometer on the FC-51 adjusts the detection threshold. Turning it clockwise increases sensitivity (larger range), counter-clockwise decreases it. For bin-mouth detection, it should be set to about 10–20 cm range so it triggers reliably when a hand approaches but does not false-trigger from objects across the room.



2.4 SSD1306 0.96" OLED Display

The SSD1306 is a compact monochrome OLED display with 128x64 pixel resolution that communicates over the I2C bus. It provides real-time local feedback — a critical feature for a system deployed away from a PC.

Role & Functionality in This Project

The OLED acts as the local user interface of the entire system. Without it, the only way to check the bin status would be to open a laptop and look at ThingSpeak. The display gives instant on-device feedback:

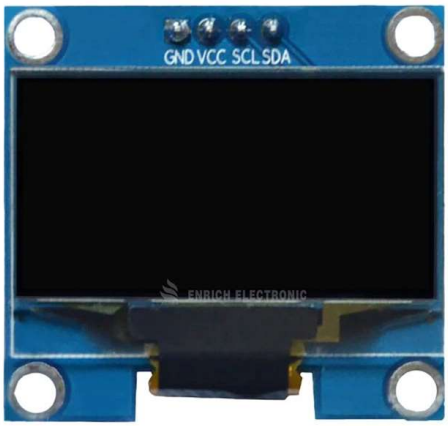
- Title Bar — Displays "WASTE MONITOR" at the top, followed by a horizontal divider line — clearly identifies the device at a glance.
- Large Fill Percentage — The fill percentage is rendered in 3x large text (e.g., "86%") in the center of the screen, readable from several meters away.
- Visual Fill Bar — A proportional progress bar beneath the percentage gives an intuitive graphical representation of bin fill level, similar to a battery indicator.
- Distance Reading — Shows the raw ultrasonic distance (e.g., "Dist: 5.6cm") at the bottom-left for diagnostic purposes.
- IR Status — Shows "IR: YES" or "IR: NO" at the bottom-right, confirming presence detection state.
- Wi-Fi Status Screen — During startup, the OLED shows "Connecting to WiFi..." and then "WiFi Connected! [IP address]" before entering the main display loop.

Why OLED was chosen over LCD: OLEDs have self-emitting pixels (no backlight needed), giving them extremely high contrast, wide viewing angles, and very low power consumption compared to LCD displays. They are also physically thinner and visually sharper, making them well-suited for compact IoT devices. The SSD1306 specifically is the most commonly used small display in Arduino/ESP8266 projects due to its excellent library support (Adafruit_SSD1306).

Technical Specifications:

- Operating Voltage: 3.3V to 5V
- Interface: I2C (I2C address 0x3C)
- Resolution: 128 x 64 pixels (monochrome white-on-black)
- Display Technology: OLED (Organic Light Emitting Diode) — no backlight
- Driver IC: SSD1306
- Viewing Angle: >160 degrees
- Power Consumption: ~20 mA active, <1 mA sleep
- Connections: VCC (3.3V), GND, SCL (D1/GPIO5), SDA (D2/GPIO4)

I2C Communication: The OLED uses the I2C protocol at address 0x3C. The Wire library manages the I2C bus, and the Adafruit_SSD1306 library abstracts the display into high-level drawing commands (setCursor, print, drawRect, fillRect, drawLine). The display buffer is written to RAM and then pushed to the physical display with a single display.display() call each loop.



2.5 ThingSpeak — IoT Cloud Platform

ThingSpeak is a cloud-based IoT analytics platform by MathWorks (the makers of MATLAB). It provides a free-to-use HTTP API for collecting, storing, and visualizing time-series sensor data from IoT devices.

<u>Role & Functionality in This Project</u>

ThingSpeak is the remote monitoring and data logging backbone of the project. Without it, the system would only have local display capability and no historical record:

- Remote Monitoring — Anyone with access to the channel can view live and historical bin data from any device with a browser, anywhere in the world — no physical access to the device needed.
- Time-Series Logging — Every data upload is timestamped and stored permanently. This allows analysis of how bin fill levels change throughout the day, week, or over time.
- Three-Channel Data Dashboard — Fields 1 (Fill %), Field 2 (Distance cm), and Field 3 (IR Binary) are each plotted as separate charts, giving a full data picture.
- API-Based Upload — The NodeMCU uses the ThingSpeak Arduino library, which sends an HTTP POST request with the field values to the ThingSpeak REST API. The channel ID (3279204) and Write API Key authenticate the upload.

- Rate-Limiting Awareness — ThingSpeak enforces a minimum 15-second interval between updates on free accounts. The project code uses millis() to track time since the last upload and only sends data when 15 seconds have elapsed — strictly complying with this constraint.

Why ThingSpeak: It offers a free tier that is perfectly suited for student/hobbyist IoT projects, requires no server setup, has a ready-made visualization dashboard, and has an officially maintained Arduino library. The MATLAB integration also allows advanced data analysis if needed.

3. Working Principles of Sensors

3.1 HC-SR04 Ultrasonic Sensor — Working Principle

Principle: Time-of-Flight (ToF) Measurement

The HC-SR04 operates on the principle of echolocation — exactly like a bat or a submarine sonar. It emits a burst of ultrasonic sound (at 40 kHz, far above human hearing range of 20 kHz) and measures the time taken for the sound echo to return after reflecting off the target surface.

Step-by-step Operational Sequence:

- Step 1 — The microcontroller sends a 10 microsecond HIGH pulse to the TRIG pin.
- Step 2 — The sensor internally triggers a burst of 8 ultrasonic pulses at 40 kHz.
- Step 3 — Immediately after, the ECHO pin goes HIGH.
- Step 4 — The 40 kHz sound waves travel outward at the speed of sound (~343 m/s at room temp).
- Step 5 — Waves hit the waste surface (or any obstacle) and reflect back.
- Step 6 — When the echo is received by the sensor, the ECHO pin goes LOW.
- Step 7 — The total duration the ECHO pin was HIGH = round-trip travel time of sound.

Core Formula — Distance Calculation

Since the sound travels TO the object and BACK, the total path is twice the actual distance:

Total Path = Speed of Sound x Time Elapsed

Distance = (Speed of Sound x Echo Duration) / 2

Speed of sound in air at ~25 degrees C = 343 m/s = 0.0343 cm/microsecond. Substituting:

Distance (cm) = (Echo_Duration_microseconds x 0.0343) / 2

Implementation in project code:

```
digitalWrite(TRIG_PIN, LOW);   delayMicroseconds(2);
digitalWrite(TRIG_PIN, HIGH);  delayMicroseconds(10);
digitalWrite(TRIG_PIN, LOW);
long duration = pulseIn(ECHO_PIN, HIGH, 30000); // 30ms timeout
float distance = (duration * 0.0343) / 2.0;
```

The 30,000 microsecond timeout in pulseIn() ensures the MCU does not freeze if no echo returns (e.g., sensor aimed at a surface too far away or in open air). If timeout occurs, duration = 0 and the function returns 0 cm.

Fill Percentage Calculation Formula

With the bin height defined as 40 cm (BIN_HEIGHT_CM = 40), the fill percentage is derived from the measured distance. Since the sensor is at the top, less distance = more waste:

Fill% = ((BIN_HEIGHT - Distance) / BIN_HEIGHT) x 100

Examples from actual Serial Monitor output:

- Distance = 36.20 cm → Fill = ((40 - 36.20) / 40) x 100 = 9% (nearly empty)
- Distance = 17.97 cm → Fill = ((40 - 17.97) / 40) x 100 = 55% (half full)
- Distance = 5.57 cm → Fill = ((40 - 5.57) / 40) x 100 = 86% (nearly full)

The result is constrained between 0% and 100% using constrain(percent, 0, 100) to handle edge cases.

3.2 IR Proximity Sensor — Working Principle

Principle: Active Infrared Reflection

The IR proximity sensor uses the principle of active infrared reflection detection. An IR LED on the module continuously emits infrared radiation (typically at 940 nm wavelength). A paired IR photodiode detects any reflected IR energy from nearby objects. The received signal is fed into a comparator circuit (usually an LM393) that compares the received intensity against a threshold.

Step-by-step Operational Sequence:

- Step 1 — The IR LED emits a continuous cone of infrared light forward.
- Step 2 — When no object is nearby, IR light dissipates into space — no reflection.
- Step 3 — When an object (hand, waste, lid) enters the range, IR light reflects back toward the module.
- Step 4 — The IR photodiode picks up the reflected energy.
- Step 5 — The onboard comparator compares detected intensity against the threshold set by the potentiometer.
- Step 6 — If intensity exceeds threshold: OUT pin goes LOW (Active LOW logic — object detected).
- Step 7 — If no reflection: OUT pin stays HIGH (clear).

Digital Detection Logic (Active LOW)

The IR sensor uses Active LOW logic: the output pin reads LOW when an object is present and HIGH when the path is clear. This inverted logic is handled in code as:

```
// LOW means object detected (active-low logic)
irDetected = (digitalRead(IR_PIN) == LOW);

// On OLED: displayed as "IR: YES" or "IR: NO"
// On ThingSpeak: uploaded as 1 (detected) or 0 (clear)
ThingSpeak.setField(3, ir ? 1 : 0);
```

Key Characteristics of the IR Sensor

- Non-contact detection — no physical touch needed
- Binary output only — detects presence/absence, NOT distance
- Very fast response time (microseconds)
- Adjustable range via onboard trimpot (2 to 30 cm)
- Sensitive to ambient IR (sunlight can cause false positives outdoors)
- Works best for short-range detection at bin mouth level

IR vs Ultrasonic — Comparison

Feature	IR Sensor (FC-51)	Ultrasonic (HC-SR04)
Output	Digital (binary)	Analog duration → distance
Use in project	Presence detection at bin mouth	Fill level measurement
Range	2–30 cm (adjustable)	2–400 cm
Accuracy	N/A (binary only)	+/- 3 mm
Interference	Affected by sunlight	Affected by soft/angled surfaces
Power (NodeMCU)	D5 (GPIO14)	D6 (TRIG), D7 (ECHO)

4. System Architecture & Data Flow

The system follows a continuous sense-process-display-upload loop executing every ~1 second in the main loop():

- Sensor Read — HC-SR04 fires TRIG pulse; Echo duration measured; IR digital state read.
- Processing — Raw distance converted to fill %; IR state interpreted (active-low logic).
- OLED Update — Display refreshed every loop iteration with current values.
- Cloud Upload — Every 15 seconds, 3 fields pushed to ThingSpeak via Wi-Fi (HTTP).
- Serial Log — Continuous serial output at 115200 baud for debugging.

Wi-Fi & Cloud:

The ESP8266 connects to the configured Wi-Fi network on startup (connectWiFi() in setup()). If disconnected during operation, it automatically reconnects before the next ThingSpeak upload. The ThingSpeak library handles the HTTP POST request to write field data to the cloud channel.

5. Code & Implementation

5.1 Libraries and Pin Definitions

```
#include <Wire.h>            // I2C for OLED
#include <Adafruit_GFX.h>    // OLED graphics
#include <Adafruit_SSD1306.h>// OLED driver
#include <ESP8266WiFi.h>     // Wi-Fi
#include <ThingSpeak.h>      // Cloud upload

#define TRIG_PIN  D6   // HC-SR04 Trigger  → GPIO12
#define ECHO_PIN  D7   // HC-SR04 Echo     → GPIO13
#define IR_PIN    D5   // IR Sensor OUT    → GPIO14
#define BIN_HEIGHT_CM 40  // Bin height in cm
```

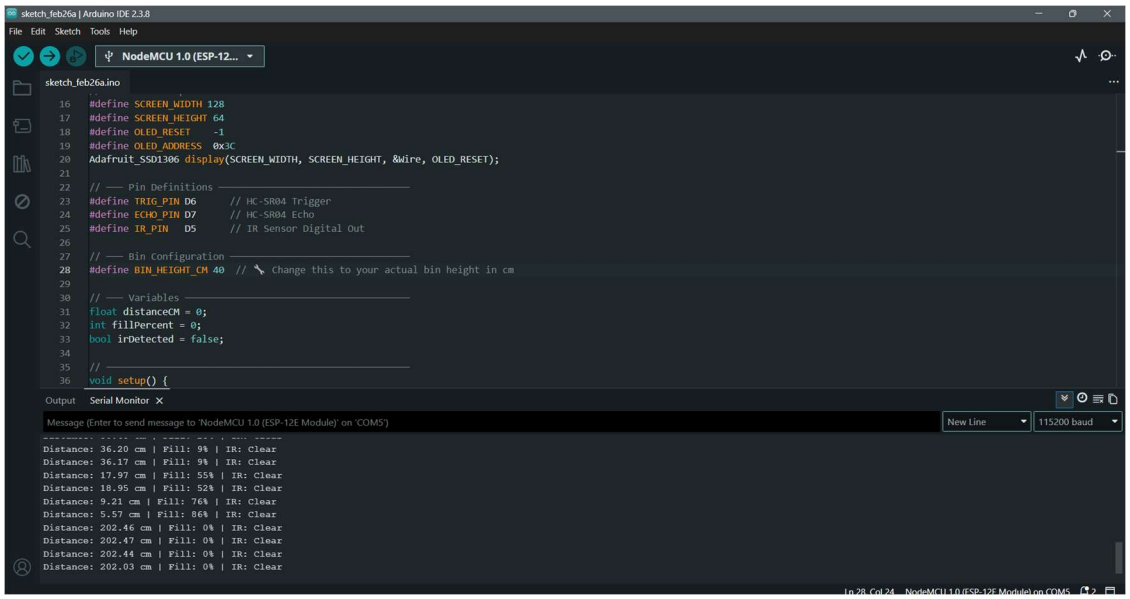


Figure 2: Arduino IDE — Pin definitions, OLED setup, and Serial Monitor during live test run

5.2 Distance Measurement Function

```
float measureDistance() {
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);          // 10us trigger pulse
  digitalWrite(TRIG_PIN, LOW);
  long duration = pulseIn(ECHO_PIN, HIGH, 30000); // 30ms timeout
  if (duration == 0) return 0;    // No echo received
  return (duration * 0.0343) / 2.0; // Convert to cm
}
```

5.3 Fill Percentage Calculation

```
int calculateFillPercent(float distance) {
  if (distance >= BIN_HEIGHT_CM) return 0;   // Out of range = empty
  if (distance <= 0) return 100;              // 0 distance = full
  int percent = (int)(((BIN_HEIGHT_CM - distance)
                        / BIN_HEIGHT_CM) * 100);
  return constrain(percent, 0, 100);
}
```

5.4 ThingSpeak Upload

Data is pushed to 3 ThingSpeak fields every 15 seconds:

```
ThingSpeak.setField(1, fill);           // Fill percent
ThingSpeak.setField(2, dist);           // Distance cm
ThingSpeak.setField(3, ir ? 1 : 0);    // IR binary
int result = ThingSpeak.writeFields(channelID, writeAPIKey);
```

6. System Outputs & Results

6.1 Serial Monitor Output

The NodeMCU continuously logs sensor data at 115200 baud every second, plus ThingSpeak upload results:

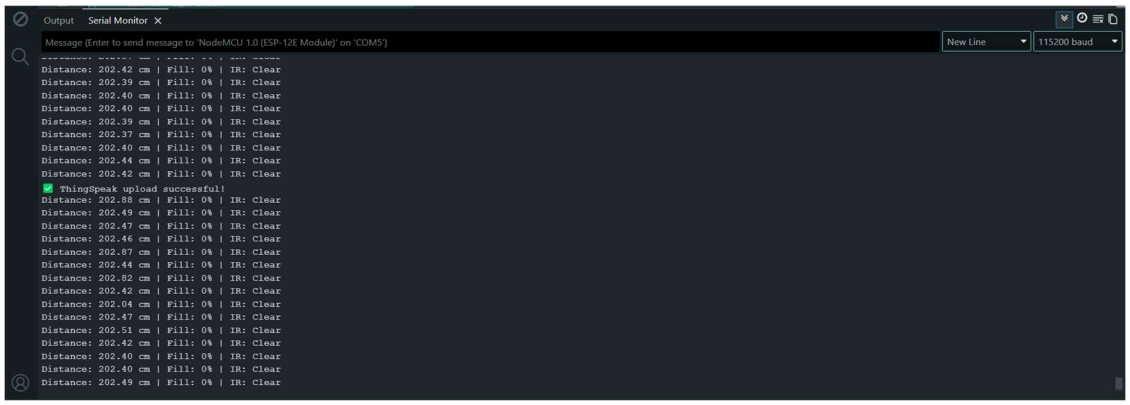


Figure 3: Serial Monitor — Live distance, fill %, IR status, and ThingSpeak upload confirmation messages

6.2 ThingSpeak Cloud Dashboard

All data is uploaded to ThingSpeak Channel ID 3279204 every 15 seconds (mandatory minimum interval). Three fields are tracked:

Field	Data	Description
Field 1	Fill Percent (%)	Waste level as % of bin height
Field 2	Distance (cm)	Raw ultrasonic sensor reading in cm
Field 3	IR Detected (0 or 1)	Binary: 1 = object detected, 0 = clear

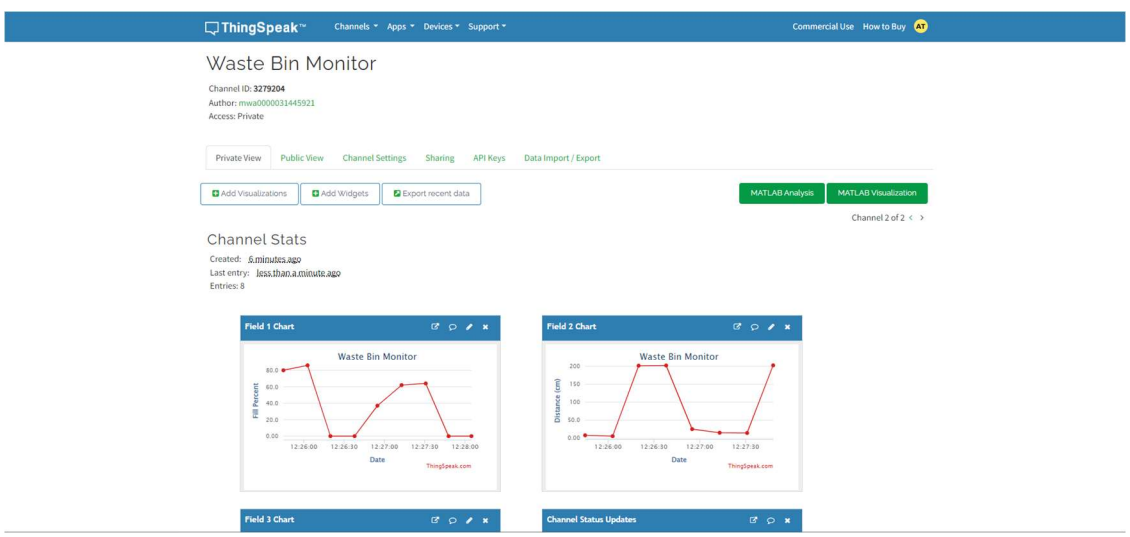


Figure 4: ThingSpeak Dashboard — Channel overview with Field 1 (Fill %) and Field 2 (Distance cm) time-series charts

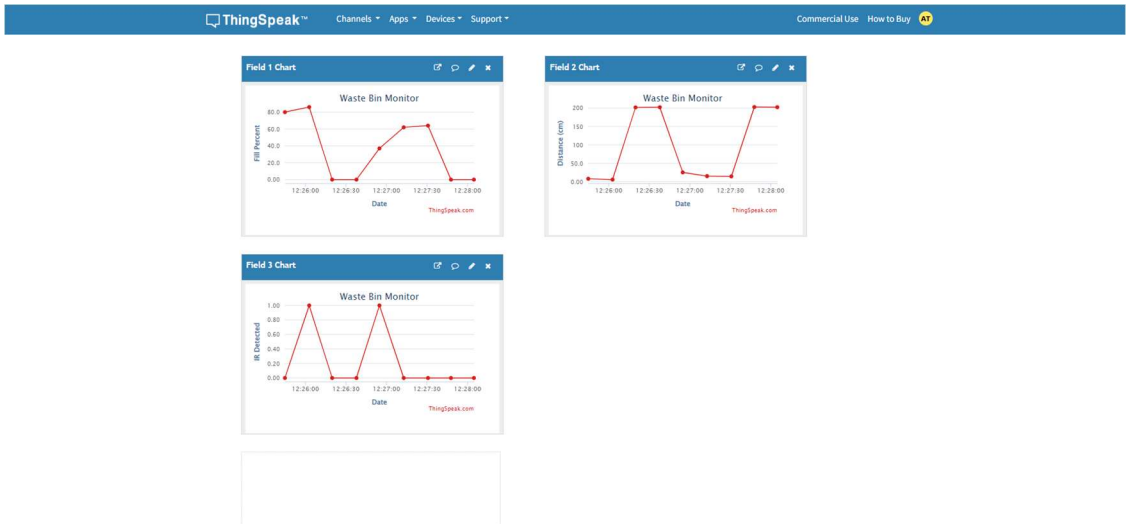


Figure 5: ThingSpeak — Field 1 (Fill %), Field 2 (Distance cm), and Field 3 (IR Detected) charts over time

The Field 1 chart shows fill percentage varying as objects are placed in and removed from the bin. Field 2 shows the inverse relationship — as fill increases, distance decreases. Field 3 shows binary IR detection events (0 or 1).

6.3 Physical Hardware Setup

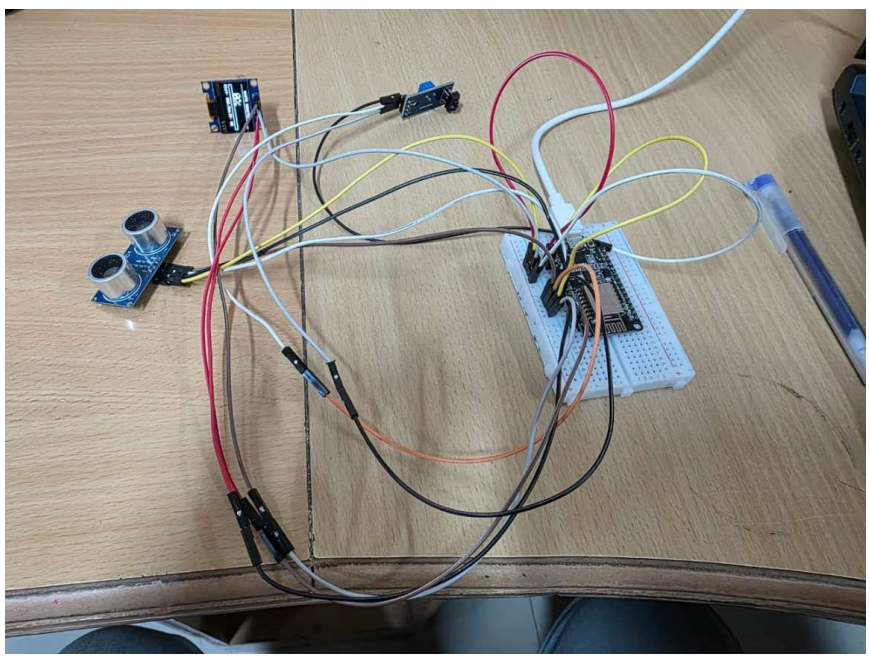


Figure 6: Top-down view of complete assembled circuit — HC-SR04 (bottom-left), IR sensor (top-center), OLED (top-left), NodeMCU on breadboard (right)

7. Complete Summary Table

Parameter	Details
Microcontroller	NodeMCU v1.0 — ESP8266 ESP-12E @ 80 MHz
Distance Sensor	HC-SR04 Ultrasonic (2–400 cm, accuracy ±3 mm)
Presence Sensor	IR Proximity FC-51 (2–30 cm, digital output)
Display	SSD1306 0.96" OLED 128x64 via I2C (0x3C)
Cloud Platform	ThingSpeak — Channel ID 3279204
Upload Interval	Every 15 seconds (ThingSpeak minimum)
TRIG Pin (HC-SR04)	D6 / GPIO12
ECHO Pin (HC-SR04)	D7 / GPIO13
IR Sensor Pin	D5 / GPIO14
OLED I2C (SCL/SDA)	D1 (GPIO5) / D2 (GPIO4)
Distance Formula	Distance (cm) = (Duration_us x 0.0343) / 2
Fill Formula	Fill% = ((BIN_HEIGHT - Distance) / BIN_HEIGHT) x 100
Bin Height (configurable)	40 cm (BIN_HEIGHT_CM define)
Serial Baud Rate	115200
Programming IDE	Arduino IDE 2.3.8

The IoT Waste Bin Monitor successfully demonstrates integration of ultrasonic ranging, infrared proximity detection, OLED display, and cloud IoT data publishing on the ESP8266 platform. The distance formula (Duration x 0.0343 / 2) and fill formula ((BIN_HEIGHT - Distance) / BIN_HEIGHT x 100) form the mathematical core of the system, enabling accurate and real-time waste level monitoring.